

CSS - Cascading Style Sheets

1. Introduction

CSS est utilisé pour définir les styles de vos pages Web, y compris la conception, la mise en page et les variations d'affichage pour différents appareils et tailles d'écran.

Les instructions CSS peuvent être écrites :

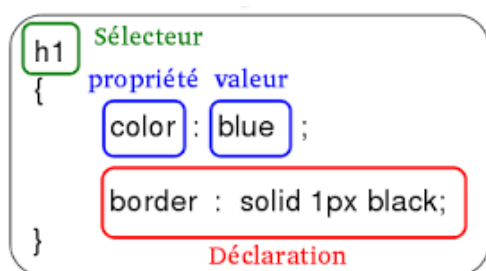
- Dans les balises concernées (En utilisant **l'attribut style**)
- Dans l'en-tête du document (on parle de feuille de styles interne). En utilisant **la balise < style >**
- Dans un fichier séparé (on parle de feuille de styles externe). En utilisant **la balise <link>**

Note :

- Un fichier HTML peut référencer plusieurs fichiers CSS.
- Il vaut mieux mettre les feuilles de style dans des fichiers externes pour faciliter la maintenance.

2. Syntaxe CSS

Une règle CSS se compose d'un sélecteur et d'un bloc de déclaration, ce dernier contient une ou plusieurs déclarations séparées par des points-virgules.



Note : Les sélecteurs CSS sont sensibles à la casse

3. Feuilles de style multiples

- Si certaines propriétés ont été définies pour le même sélecteur (élément) dans différentes feuilles de style, **la valeur de la dernière feuille de style lue sera utilisée.**
- Si vous affectez trois valeurs différentes à une même propriété CSS, le code CSS écrit dans une balise écrase celui qui est défini dans l'en-tête du document et celui qui est défini dans une feuille de styles externe.

4. Sélecteurs CSS

Les **sélecteurs CSS** sont utilisés pour **cibler les éléments HTML** que vous souhaitez mettre en forme. Nous pouvons les diviser en **cinq catégories**:

1. **Sélecteurs simples** : sélectionnez les éléments en fonction du nom, de l'identifiant, de la classe
2. **Sélecteurs de combineur** : sélectionnez les éléments en fonction d'une relation spécifique entre eux
3. **Sélecteurs de pseudo-classes** : sélectionnez des éléments en fonction d'un certain état
4. **Sélecteurs de pseudo-éléments** : sélectionner et styliser une partie d'un élément
5. **Sélecteurs d'attributs** : sélectionnez des éléments en fonction d'un attribut ou d'une valeur d'attribut

4.1. Sélecteurs simples

Sélecteur	Exemple	Description de l'exemple
#id	#unique	Sélectionne l'élément avec id = "unique"
.class	.intro	Sélectionne tous les éléments avec class = "intro"
element.class	p.intro	sélectionne uniquement les éléments <p> avec class = "intro"
*	*	Sélectionne tous les éléments
Element	p	Sélectionne tous les éléments <p>
element,element,..	div, p	Sélectionne tous les éléments <div> et tous les éléments <p>

4.2. Sélecteurs de combinateur

Sélecteurs	Exemple	Description de l'exemple
1 - Descendant Selector: (Descendants indirects)	div p	Sélectionne tous les éléments <p> à l'intérieur des éléments <div>
2 - Child Selector: (Descendant direct)	div > p	Sélectionne tous les éléments <p> où le parent est un élément <div> (enfants directs)
3 - Adjacent Sibling Selector: (Frère direct)	div + p	Sélectionne le premier élément <p> placé immédiatement après les éléments <div>
4 - General Sibling Selector: (Frère indirect)	div ~ p	Sélectionne chaque élément <p> précédé d'un élément <div>

4.3. Sélecteurs de pseudo-classes

Une **pseudo-classe** est utilisée pour **définir un état spécial d'un élément**. Il peut être utilisé pour:

- Styliser un élément lorsqu'un utilisateur passe la souris dessus
- Style différent des liens visités et non visités
- Styliser un élément lorsqu'il obtient le focus
- ...etc.

La **syntaxe des pseudo-classes**:

selector:**pseudo-class** { property: value; }

Sélecteur	Exemple	Description de l'exemple
:active	a:active	Sélectionne le lien actif
:checked	input:checked	Sélectionne chaque élément <input> coché
:disabled	input:disabled	Sélectionne chaque élément <input> désactivé
:empty	p:empty	Sélectionne chaque élément <p> qui n'a pas d'enfants
:enabled	input:enabled	Sélectionne chaque élément <input> activé
:first-child	p:first-child	Sélectionne tous les éléments <p> qui sont le premier enfant de son parent
:first-of-type	p:first-of-type	Sélectionne chaque élément <p> qui est le premier élément <p> de son parent
:focus	input:focus	Sélectionne l'élément <input> qui a le focus
:hover	a:hover	Sélectionne les liens au survol de la souris
:in-range	input:in-range	Sélectionne les éléments <input> avec une valeur dans une plage spécifiée
:invalid	input:invalid	Sélectionne tous les éléments <input> avec une valeur non valide
:lang(language)	p:lang(it)	Selects every <p> element with a lang attribute value starting with "it"
:last-child	p:last-child	Sélectionne tous les éléments <p> qui sont le dernier enfant de son parent
:last-of-type	p:last-of-type	Sélectionne chaque élément <p> qui est le dernier élément <p> de son parent
:link	a:link	Sélectionne tous les liens non visités
:not(selector)	:not(p)	Sélectionne chaque élément qui n'est pas un élément <p>
:nth-child(n)	p:nth-child(2)	Sélectionne chaque élément <p> qui est le deuxième enfant de son parent
:nth-last-child(n)	p:nth-last-child(2)	Sélectionne chaque élément <p> qui est le deuxième enfant de son parent, à partir du dernier enfant
:nth-last-of-type(n)	p:nth-last-of-type(2)	Sélectionne chaque élément <p> qui est le deuxième élément <p> de son parent, à partir du dernier enfant
:nth-of-type(n)	p:nth-of-type(2)	Sélectionne chaque élément <p> qui est le deuxième élément <p> de son

		parent
:only-of-type	p:only-of-type	Sélectionne chaque élément <p> qui est le seul élément <p> de son parent
:only-child	p:only-child	Sélectionne chaque élément <p> qui est le seul enfant de son parent
:optional	input:optional	Sélectionne les éléments <input> sans attribut "required"
:out-of-range	input:out-of-range	Sélectionne les éléments <input> avec une valeur en dehors d'une plage spécifiée
:read-only	input:read-only	Sélectionne les éléments <input> avec un attribut "readonly"
:read-write	input:read-write	Sélectionne les éléments <input> sans attribut "readonly"
:required	input:required	Sélectionne les éléments <input> avec un attribut "required"
:root	Root	Sélectionne l'élément racine du document
:target	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)
:valid	input:valid	Sélectionne tous les éléments <input> avec une valeur valide
:visited	a:visited	Sélectionne tous les liens visités

4.4. Sélecteurs de pseudo éléments

Un **pseudo-élément CSS** est utilisé pour styliser les **parties spécifiées d'un élément**. Il peut être utilisé pour :

- Style de la première lettre, ou ligne, d'un élément
- Insérer du contenu avant ou après le contenu d'un élément
- ...etc.

Sélecteur	Exemple	Description de l'exemple
::after	p::after	Insérer du contenu (texte ou image) après chaque élément <p>
::before	p::before	Insérer du contenu (texte ou image) avant chaque élément <p>
::first-letter	p::first-letter	Ajouter un style spécial à la première lettre de chaque élément <p>
::first-line	p::first-line	Ajouter un style spécial à la première ligne de chaque élément <p>
::selection	p::selection	Ajouter un style spécial à la partie d'un élément qui est sélectionnée par un utilisateur
::marker	::marker	sélectionne le marqueur d'un élément de liste.

Note :

La notation **::** a remplacé la notation **:** pour les pseudo-éléments dans CSS3. Il s'agissait d'une tentative du W3C de faire la distinction entre les **pseudo-classes** et les **pseudo-éléments**.

4.5. Sélecteurs d'attributs CSS

Il est possible de styliser des éléments HTML qui ont des attributs ou des valeurs d'attribut spécifiques.

- Le **sélecteur [attribute]** est utilisé pour sélectionner des éléments avec **un attribut spécifié**.
- Le **sélecteur [attribute="value"]** est utilisé pour sélectionner des éléments avec **un attribut et une valeur spécifiés**.
- Le **sélecteur[attribute~="value"]** est utilisé pour sélectionner des éléments avec **une valeur d'attribut contenant un mot spécifié**.
- Le **sélecteur [attribute|= "value"]** est utilisé pour sélectionner des éléments **avec l'attribut spécifié en commençant par la valeur spécifiée**. (La valeur doit être un mot entier, soit seul, comme class = "top", soit suivi d'un tiret (-), comme class = "top-text" !)
- Le **sélecteur [attribute^="value"]** est utilisé pour sélectionner les éléments **dont la valeur d'attribut commence par une valeur spécifiée**. (la valeur ne doit pas nécessairement être un mot entier!)
- Le **sélecteur [attribute\$="value"]** est utilisé pour sélectionner les éléments **dont la valeur d'attribut se termine par une valeur spécifiée**. (la valeur ne doit pas nécessairement être un mot entier!)
- Le **sélecteur [attribute*="value"]** est utilisé pour sélectionner des éléments **dont la valeur d'attribut contient une valeur spécifiée**. (la valeur ne doit pas nécessairement être un mot entier!)

5. Arrière- plans CSS

5.1. Couleur d'arrière-plan

Propriété	Description
background-color	spécifie la couleur d'arrière-plan d'un élément.
Opacity ¹	spécifie l'opacité / la transparence d'un élément. Il peut prendre une valeur comprise entre 0,0 et 1,0. La valeur la plus basse, la plus transparente.

Note:

1- lorsque vous utilisez la propriété **opacity** pour ajouter de la transparence à l'arrière-plan d'un élément, **tous ses éléments enfants héritent de la même transparence. Cela peut rendre le texte à l'intérieur d'un élément entièrement transparent difficile à lire.**

- ✓ Si vous ne souhaitez pas appliquer d'opacité aux éléments enfants, utilisez les valeurs de couleur **RGBA** (rouge, vert, bleu, **alpha**).

Le paramètre **alpha** est un nombre compris entre **0,0 (entièrement transparent) et 1,0 (entièrement opaque)**.

5.2. Image d'arrière-plan

Propriété	Exemple	Description
background-image	background-image: url("paper.gif");	spécifie une image à utiliser comme arrière-plan d'un élément (body, p,...)
background-repeat	background-repeat: repeat-y; (resp.background-repeat: repeat-x;)	pour répéter une image verticalement. (resp. horizontalement)
background-position	background-position: right top;	utilisée pour spécifier la position de l'image d'arrière-plan
background-attachment	background-attachment: fixed; (background-attachment: scroll;)	spécifie si l'image d'arrière-plan doit défiler ou être fixe (ne défilera pas avec le reste de la page)

Pour raccourcir le code, il est également possible de spécifier toutes les propriétés d'arrière-plan dans une seule propriété. C'est ce qu'on appelle une propriété abrégée.

Au lieu d'écrire: `body { background-color: #ffffff; background-image: url("img.png"); background-repeat: no-repeat; background-position: right top; }` Vous pouvez utiliser la propriété abrégée :
`background: body { background: #ffffff url("img.png") no-repeat right top; }`

Note: Lors de l'utilisation de la propriété abrégée, l'ordre des valeurs de propriété est: background-color, background-image, background-repeat, background-attachment, background-position.

5.3. Le dégradés en CSS

Les dégradés CSS vous permettent d'afficher des transitions fluides entre deux ou plusieurs couleurs spécifiées. CSS définit deux types de dégradés: **linéaires et radiaux**.

- **Dégradés linéaires (descend / haut / gauche / droite / diagonale):**

Vous devez définir **au moins deux couleurs**. Vous pouvez également définir un point de départ et **une direction** (ou un angle) avec l'effet de dégradé.

Syntaxe :

background-image: linear-gradient(*direction, color-stop1, color-stop2, ...*);

Exemples :

background-image: linear-gradient (to right, blue , pink);

background-image: linear-gradient (to bottom right, red, yellow, green);

Note : La fonction **repeating-radial-gradient ()** est utilisée pour répéter des dégradés linéaires.

Exemple :

background-image: repeating-linear-gradient(50deg,blue 15%, pink 20%);

- **Dégradés radiaux (définis par leur centre):**

Un dégradé radial est défini par son centre. Pour créer un dégradé radial, vous devez également définir au moins deux arrêts de couleur.

Syntaxe :

background-image: radial-gradient(*shape size at position, start-color, ..., last-color*);

Exemples :

background-image: radial-gradient(red, yellow, green);

background-image: radial-gradient(red 5%, yellow 15%, green 60%);

Note : La fonction **repeating-radial-gradient ()** est utilisée pour répéter les dégradés radiaux.

Exemple:

background-image: repeating-radial-gradient(red, yellow 10%, green 15%);

6. Les bordures en CSS

Les propriétés de bordure CSS vous permettent de spécifier le style, la largeur et la couleur de la bordure d'un élément.

Propriété	Description
border-style	spécifie le type de bordure à afficher.
border-width	spécifie la largeur des quatre bordures. La largeur peut être définie en px, pt, cm, em, etc. ou en utilisant l'une des trois valeurs prédéfinies: thin, medium, ou thick.
border-color	utilisée pour définir la couleur des quatre bordures en spécifiant un nom de couleur ou bien une valeur HEX, RVB ou HSL
border-radius	utilisée pour ajouter des bordures arrondies à un élément

Les valeurs suivantes sont autorisées pour la propriété **border-style**:

<code>border-style: dotted;</code>	<code>border-style: ridge;</code>
<code>border-style: dashed;</code>	<code>border-style: inset;</code>
<code>border-style: solid;</code>	<code>border-style: outset;</code>
<code>border-style: double;</code>	<code>border-style: none;</code>
<code>border-style: groove;</code>	<code>border-style: hidden;</code>

- La propriété **border-style** peut avoir de **une à quatre valeurs** (pour la bordure supérieure, la bordure droite, la bordure inférieure et la bordure gauche).
- En CSS, il existe également des propriétés pour spécifier chacune des bordures : **border-top-style, border-right-style, border-bottom-style, border-left-style**.
- La propriété **border-width** (resp. **border-color**) peut avoir de **une à quatre valeurs** (pour la bordure supérieure, la bordure droite, la bordure inférieure et la bordure gauche).

Exemples :

border-width: 20px 5px; pour le top et bottom .

border-width: 25px 10px 4px 35px; pour le top, right, bottom et left.

border-color: red green blue yellow;

Note :

- Pour raccourcir le code, il est possible de spécifier **toutes les propriétés** de bordure individuelles **dans une seule propriété** : « **border** ».
- Vous pouvez également spécifier toutes les propriétés de bordure individuelles pour **un seul côté**.

7. Les Marges en CSS

Les **marges** sont utilisées pour **créer un espace autour des éléments**, en dehors de toute bordure définie.

Avec CSS, vous avez un contrôle total sur les marges. Il existe des propriétés pour définir **la marge de chaque côté** d'un élément : **margin-top**, **margin-right**, **margin-bottom** et **margin-left** (haut, droite, bas et gauche).

Toutes les propriétés de marge peuvent avoir les valeurs suivantes:

- **auto** - le navigateur calcule la marge pour centrer horizontalement l'élément dans son conteneur.
- **length** - spécifie une marge en px, pt, cm, etc.
- **%** - spécifie une marge en% de la largeur de l'élément contenant
- **inherit** - spécifie que la marge doit être héritée de l'élément parent

Pour raccourcir le code, il est possible de spécifier toutes les propriétés de marge dans une seule propriété: **margin**.

8. Remplissage CSS

Le remplissage est utilisé pour créer un espace autour du contenu d'un élément, à l'intérieur de toutes les bordures définies, en utilisant la propriété **Padding**.

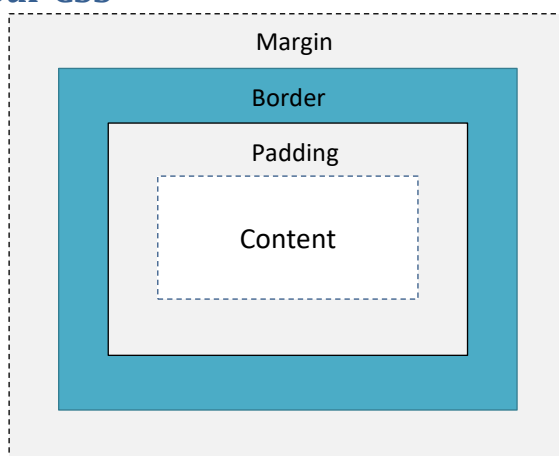
Il existe des propriétés pour définir le remplissage de chaque côté d'un élément : **padding-top**, **padding-right**, **padding-bottom** et **padding-left** (haut, droite, bas et gauche).

Toutes ces propriétés peuvent avoir les valeurs suivantes:

- **length** - spécifie un remplissage en px, pt, cm, etc.
- **%** - spécifie un remplissage en% de la largeur de l'élément contenant.
- **inherit** - spécifie que le remplissage doit être hérité de l'élément parent.

Remarque: les valeurs négatives ne sont pas autorisées.

9. Hauteur et largeur CSS



- Les propriétés **height** et **width** sont utilisés pour définir la hauteur et la largeur d'un élément.
- La propriété **max-width** est utilisée pour définir la largeur maximale d'un élément.

Les propriétés height et width peuvent avoir les valeurs suivantes:

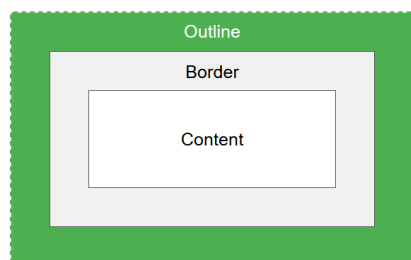
- **auto** - C'est la valeur par défaut. Le navigateur calcule la hauteur et la largeur
- **length** - Définit la hauteur / largeur en px, cm etc.
- **%** - Définit la hauteur / largeur en pourcentage du bloc contenant
- **initial** - Règle la hauteur / largeur à sa valeur par défaut
- **inherit** - La hauteur / largeur sera héritée de sa valeur parent

Note:

- Les propriétés de **height** et de **width** n'incluent pas le remplissage, les bordures ou les marges.
- **La hauteur et la largeur totale d'un élément** doit être calculée comme ceci:
 - La Largeur totale de l'élément = largeur + remplissage gauche + remplissage droit + bordure gauche + bordure droite + marge gauche + marge droite.
 - Hauteur totale de l'élément = hauteur + remplissage supérieur + remplissage inférieur + bordure supérieure + bordure inférieure + marge supérieure + marge inférieure

10. Le contour en CSS

Un contour est une ligne tracée à l'extérieur de la bordure de l'élément.



CSS a les propriétés de contour suivantes: **outline-style**, **outline-color**, **outline-width**, **outline-offset**, **outline**.

Note:

- La propriété **outline-style** doit être définie, afin que les autres propriétés de contour aient un effet.
 - Le contour **ne fait PAS partie des dimensions de l'élément**; la largeur et la hauteur totales de l'élément ne sont pas affectées par la largeur du contour.
- La propriété **outline-width** spécifie la largeur du contour et peut avoir l'une des valeurs suivantes: thin (généralement 1px), medium (généralement 3px), thick (généralement 5px), ou bien une taille spécifique (en px, pt, cm, em, etc)
 - La propriété **outline-color** est utilisée pour définir la couleur du contour en utilisant: le nom de la couleur, la valeur hexadécimale HEX, la valeur RVB, la valeur HSL ou inverser (effectue une inversion de couleur garantissant que le contour est visible, quelle que soit la couleur de fond)
 - La propriété **outline** est une propriété abrégée permettant de définir les propriétés de contour individuelles suivantes: outline-width, outline-style (obligatoire), outline-color.
 - La propriété **outline-offset** ajoute un espace entre un contour et la bordure d'un élément. L'espace entre un élément et son contour est **transparent**.

11. Mise en forme du texte en CSS

CSS a beaucoup de propriétés pour la mise en forme du texte.

Propriété	description
Color	utilisée pour définir la couleur du texte. Elle est spécifiée par: un nom de couleur, une valeur HEX ou bien une valeur RVB.
text-align	utilisée pour définir l'alignement horizontal d'un texte (center, left, right, justify).
vertical-align	définit l'alignement vertical d'un élément. (top, middle, bottom)
text-decoration	utilisée pour définir ou supprimer des décorations du texte. (none, overline, line-through, underline)
text-transform	utilisée pour spécifier des lettres majuscules et minuscules dans un texte (uppercase, lowercase, capitalize).
text-indent	utilisée pour spécifier l'indentation de la première ligne d'un texte.
letter-spacing	utilisée pour spécifier l'espace entre les caractères dans un texte. (valeur numérique)
line-height	utilisée pour spécifier l'espace entre les lignes.

white-space	spécifie comment les espaces blancs à l'intérieur d'un élément sont gérés. (normal nowrap pre pre-line pre-wrap initial inherit)
word-spacing	augmente ou diminue l'espace blanc entre les mots. (in px, pt, cm, em, etc).
text-shadow	ajoute une ombre au texte., en spécifiant l'ombre horizontale et l'ombre verticale.
direction	spécifie la direction du texte / la direction d'écriture dans un élément de niveau bloc.
unicode-bidi	utilisée avec la propriété direction pour définir ou renvoyer si le texte doit être remplacé pour prendre en charge plusieurs langues dans le même document.

Note :

- Si vous définissez la propriété **color**, il est recommandé de définir aussi la propriété **background-color**.
- La valeur **text-decoration: none;** est souvent utilisée pour supprimer les soulignements des liens.
- il n'est pas recommandé de souligner du texte qui n'est pas un lien

12. La police en CSS

Le choix de la bonne police a un impact énorme sur la façon dont les lecteurs découvrent un site Web.

En CSS, il existe cinq familles de polices génériques:

Tous les différents noms de polices appartiennent à l'une des familles de polices génériques :

- **Les polices Serif** ont un petit trait sur les bords de chaque lettre. Ils créent un sentiment de formalité et d'élégance.
- **Les polices sans** empattement ont des lignes épurées (pas de petits traits attachés). Ils créent un look moderne et minimaliste.
- **Polices monospace** - ici toutes les lettres ont la même largeur fixe. Ils créent un look mécanique.
- **Les polices cursives** imitent l'écriture humaine.
- **Les polices fantaisie** sont des polices décoratives / ludiques.

Propriété	Description
font-family	Utilisée pour spécifier la police d'un texte.
font-style	principalement utilisée pour spécifier du texte en italique. (normal, italic, oblique)
font-weight	spécifie le poids d'une police (normal, lighter, bold)
font-variant	spécifie si un texte doit ou non être affiché dans une police en petites majuscules. (normal, small-caps)
font-size	définit la taille du texte.

La propriété **font-family** doit contenir **plusieurs noms de polices** comme système de «secours», pour assurer une compatibilité maximale entre les navigateurs / systèmes d'exploitation. Commencez par la police souhaitée et terminez par une famille générique (pour laisser le navigateur choisir une police similaire dans la famille générique, si aucune autre police n'est disponible).

Quelque police populaire : **Arial , Verdana , Helvetica , Tahoma , Times New Roman, Géorgie, Garamond , Courier New**

- Les noms de police doivent être **séparés par une virgule**.
- Si le nom de la police comprend plusieurs mots, il doit être **entre guillemets**.
Exemple: font-family: "Times New Roman", Times, serif;
- Si vous ne spécifiez pas de taille de police, la taille par défaut du texte normal, comme les paragraphes, est 16px
- de nombreux développeurs utilisent em au lieu de pixels. (16px = 1em).
- La taille du texte peut être définie avec l'unité vw, qui signifie la "largeur de la fenêtre". De cette façon, la taille du texte suivra la taille de la fenêtre du navigateur.
La fenêtre est la taille de la fenêtre du navigateur.
1vw = 1% de la largeur de la fenêtre. Si la fenêtre a une largeur de 50 cm, 1vw est de 0,5 cm.

Note:

La propriété **font** est une propriété abrégée pour: font-style, font-variant, font-weight font-size, font-family.

Les valeurs **font-size** et **font-family** sont obligatoires. Si l'une des autres valeurs est manquante, leur valeur par défaut est utilisée.

13. Les listes en CSS

propriété	Description
list-style-type	spécifie le type de marqueur d'élément de liste. (none, circle, square, disc) pour les listes non ordonnées (upper-alpha, upper-roman, lower-alpha, lower-greek, lower-latin,...) pour listes ordonnées
list-style-image	spécifie une image comme marqueur d'élément de liste: list-style-image: url('sqpurple.gif');
list-style-position	spécifie la position des marqueurs d'élément de liste (puces) (outside, inside)
list-style	utilisé pour définir toutes les propriétés de la liste en une seule déclaration: list-style: square inside url("sqpurple.gif");

14. Les Tables en CSS

L'apparence d'un tableau HTML peut être grandement améliorée avec CSS:

propriété	Description	Exemple
Border	Pour spécifier les bordures de tableau.	table, th, td { border: 1px solid black;}
Width	Pour afficher un tableau qui va couvrir la largeur de l'écran.	table { width: 100%;}
Height	Pour définir la hauteur du tableau ou bien des éléments <th> ou <td>.	th {height: 70px;}
border-collapse	Pour réduire les bordures du tableau en une seule bordure.	table { border-collapse: collapse;}
text-align	définit l'alignement horizontal (comme à gauche, à droite ou au centre) du contenu dans <th> ou <td>.	td { text-align: center;}
vertical-align	définit l'alignement vertical (comme le haut, le bas ou le milieu) du contenu dans <th> ou <td>.	td { height: 50px; vertical-align: bottom;}
Padding	Pour contrôler l'espace entre la bordure et le contenu d'un tableau.	th, td { padding: 15px; text-align: left;}
border-bottom	Pour afficher les diviseurs horizontaux dans un tableau sans bordure	th, td { border-bottom: 1px solid #ddd;}
caption-side	Pour positionner la légende du tableau.	caption { caption-side: bottom;}

Note :

- pour mettre en surbrillance les lignes du tableau au survol de la souris. Utilisez le sélecteur **:hover** sur **<tr>** :
tr:hover {background-color:#f5f5f5;}
- Pour afficher un tableau zébré, utilisez le sélecteur **nth-child()** et ajoutez un background-color à toutes les lignes de tableau paires (ou impaires):
tr:nth-child(even) {background-color: #f2f2f2;}
- pour rendre le tableau réactif, Ajoutez un élément conteneur (comme <div>) avec **overflow-x:auto** autour de l'élément <table> :
<div style="overflow-x:auto;"> <table> ... table content ... </table> </div>

15. La propriété d'affichage en CSS

Chaque élément HTML a une valeur d'affichage par défaut : **block** ou **inline**.

- **Un élément de niveau block** commence toujours sur une nouvelle ligne et occupe toute la largeur disponible.
Exp: <div>, <h1> - <h6>, <p>, <form>, <header>, <footer>, <section>.
- **Un élément inline** ne commence pas sur une nouvelle ligne et prend seulement autant de largeur que nécessaire.
Exp: , <a>, .

Cependant, vous pouvez ignorer cela en utilisant la propriété **display**.

- La propriété **display** ne modifie **que la façon dont l'élément est affiché**, PAS le type d'élément dont il s'agit. Ainsi, un élément inline avec `display: block;` n'est pas autorisé à avoir d'autres éléments de bloc à l'intérieur.
- La propriété `display` peut avoir une de ces valeurs : **inline**, **block** et **inline-block**, la différence majeure entre eux:
 - **display: inline-block** permet de définir une largeur et une hauteur sur l'élément. De plus, les marges / rembourrages supérieurs et inférieurs sont respectés, mais pas avec **display: inline**.
 - **display: inline-block** n'ajoute pas de saut de ligne après l'élément, de sorte que l'élément peut être placé à côté d'autres éléments, pas comme **display: block**.

Note : Une utilisation courante **display: inline-block** est d'afficher les éléments de liste horizontalement au lieu de verticalement. L'exemple suivant crée des liens de navigation horizontaux.

16. Cacher un élément en CSS

Masquer un élément peut être effectué en définissant :

- ✓ la propriété **display** sur **none**; L'élément sera masqué et la page sera affichée comme si l'élément n'était pas là.
- ✓ la propriété **visibility** sur **hidden**; L'élément sera masqué mais il occupera toujours le même espace qu'auparavant.

17. La propriété de position en css

La propriété **position** spécifie le type de méthode de positionnement utilisé pour un élément. Il existe cinq valeurs de position différentes: **static**, **relative**, **fixed**, **absolute**, **sticky**.

- **static** : Un élément avec `position: static;` n'est pas positionné de manière particulière; il est toujours positionné selon le flux normal de la page. Et ils ne sont pas affectés par les propriétés `top`, `right`, `bottom` et `left`.
- **relative** : Un élément avec `position: relative;` est positionné dans le flux normal du document puis décalé, par rapport à lui-même, selon les valeurs fournies par `top`, `right`, `bottom` et `left`.
- **Fixe** : Un élément avec `position: fixed;` est positionné par rapport à la fenêtre, ce qui signifie qu'il reste toujours au même endroit même si la page défile. Les propriétés `top`, `right`, `bottom` et `left` sont utilisées pour positionner l'élément.
- **Absolue** : Un élément avec `position: Absolue;` est positionné par rapport à l'ancêtre positionné le plus proche (au lieu d'être positionné par rapport à la fenêtre, comme fixe).
- **Sticky** : Un élément avec `position: sticky;` est positionné en fonction de la position de défilement de l'utilisateur. Il bascule entre `relative` et `fixed`, en fonction de la position du défilement.

17.1. Chevauchent des éléments

Lorsque les éléments sont positionnés, ils peuvent chevaucher d'autres éléments.

La propriété **z-index** spécifie l'ordre de pile d'un élément (quel élément doit être placé devant ou derrière les autres).

Note: Un élément peut avoir un ordre d'empilement positif ou négatif.

Si le `z-index` n'est pas spécifié, l'élément positionné en dernier dans le code HTML sera affiché en haut.

17.2. Dépassement en CSS

Par défaut, le débordement est visible, ce qui signifie qu'il n'est pas tronqué et qu'il s'affiche en dehors de la boîte de l'élément: </p> <div>

La propriété **overflow** assure un meilleur contrôle de la mise en page. Elle spécifie ce qui se passe si le contenu déborde la boîte d'un élément. Elle peut avoir les valeurs suivantes:

Valeur	description
visible (Défaut)	Le débordement n'est pas écrêté. Le contenu est rendu en dehors de la boîte de l'élément
hidden	Le débordement est tronqué, et le reste du contenu sera invisible
scroll	Le débordement est tronqué, et une barre de défilement est ajoutée pour voir le reste du contenu
auto	Similaire à scroll, mais il ajoute des barres de défilement uniquement lorsque cela est nécessaire

Note : la propriété overflow ne fonctionne que pour **les éléments de bloc** avec une hauteur spécifiée.

Les propriétés **overflow-x** et **overflow-y** spécifient s'il faut gérer le débordement de contenu uniquement horizontalement ou verticalement (ou les deux).

17.3. Élément flottant en css

La propriété **float** indique qu'un élément doit être retiré du flux normal et doit être placé sur le côté droit ou sur le côté gauche de son conteneur. Dans son utilisation la plus simple, la propriété **float** peut être utilisée pour envelopper du texte autour des images.

Normalement, les éléments **div** seront affichés les uns sur les autres. Cependant, si nous utilisons, **float: left** nous pouvons laisser les éléments flotter les uns à côté des autres.

17.4. Les unités en CSS

CSS a plusieurs unités différentes pour exprimer une longueur.

Il existe deux types d'unités de longueur: **absolue** et **relative**.

- **Longueurs absolues :** Les unités de longueur absolue sont fixes (cm, mm, px, in,...). Elles ne sont pas recommandées pour une utilisation à l'écran(car les tailles d'écran varient énormément). Cependant, ils peuvent être utilisés si le support de sortie est connu, par exemple pour la mise en page d'impression.
- **Longueurs relatives :** Les unités de longueur relative spécifient une longueur par rapport à une autre propriété de longueur (em, ex, ch, vw, %,....). Les unités de longueur relative s'adaptent mieux aux différents supports de sortie.

Note:

- Un espace ne peut pas apparaître entre le numéro et l'unité. Cependant, si la valeur est 0, l'unité peut être omise.
- Pour certaines propriétés CSS, les longueurs **négatives** sont autorisées.

18. La spécificité en CSS

S'il y a deux ou plusieurs règles CSS en conflit qui pointent vers le même élément, le navigateur suit certaines règles pour déterminer laquelle est la plus spécifique.

18.1. La hiérarchie de spécificité

Il existe quatre catégories qui définissent le niveau de spécificité d'un sélecteur:

- **Styles en ligne** - Un style en ligne est directement attaché à l'élément à styliser.
Exemple: <h1 style = "color: #ffffff; ">.
- **ID** - Un ID est un identifiant unique pour les éléments de page, tel que #navbar.
- **Classes, attributs et pseudo-classes** - Cette catégorie comprend .classes, [attributs] et pseudo-classes telles que hover,: focus etc.
- **Éléments et pseudo-éléments** - Cette catégorie comprend les noms d'éléments et les pseudo-éléments, tels que h1, div,: before et: after.

18.2. Comment calculer la spécificité?

- Commencez à 0,
- ajoutez 1000 pour l'attribut de style,
- ajoutez 100 pour chaque ID,
- ajoutez 10 pour chaque attribut, classe ou pseudo-classe,
- ajoutez 1 pour chaque nom d'élément ou pseudo-élément.

Exemple :

A: h1

B: #content h1

C: <div id="content"><h1 style="color: #ffffff">Heading</h1></div>

- La spécificité de A est de 1 (un élément)
- La spécificité de B est de 101 (une référence d'identification et un élément)
- La spécificité de C est de 1000 (style en ligne)

Puisque $1 < 101 < 1000$, la troisième règle (C) a un plus haut niveau de spécificité, et sera donc appliquée.

Note :

Le sélecteur universel (*), les combinateurs (+, >, ~, ' '), et la pseudo-class de négation (:not) n'affectent en rien la spécificité.

18.3. Les règles

- Spécificité égale: la dernière règle compte.
- Les sélecteurs d'ID ont une spécificité plus élevée que les sélecteurs d'attributs.
- Un sélecteur de classe bat n'importe quel nombre de sélecteurs d'élément

18.4. La règle !Important

La règle **!important** en CSS est utilisée pour ajouter plus d'importance à une propriété / valeur qu'à la normale. En fait, si vous utilisez la règle **!important**, elle remplacera TOUTES les règles de style précédentes pour cette propriété spécifique sur cet élément!

Note : il est fortement recommandé de ne jamais utiliser cette règle, sauf en dernier recours. Car elle modifie le fonctionnement normal de la cascade, de sorte qu'il peut être très difficile de résoudre les problèmes de débogage CSS, en particulier dans une grande feuille de style.

19. Compteurs CSS

Les compteurs CSS sont des variables dont les valeurs sont incrémentées par les règles CSS. Pour travailler avec ces compteurs, nous utiliserons les propriétés et les fonctions suivantes:

La valeur d'un compteur peut être manipulée grâce aux propriétés **counter-reset**, **counter-increment** et on peut les afficher sur la page grâce aux fonctions **counter()** et **counters()** dans la propriété **content**.

Note :

- On peut définir autant de compteurs qu'on veut.
- Pour utiliser un compteur CSS, il doit d'abord être créé avec **counter-reset**.
- Un compteur peut également être utile pour créer des listes encadrées car une nouvelle instance de compteur est **automatiquement créée dans les éléments enfants**.

20. Effets d'ombre en CSS

Avec CSS, vous pouvez ajouter une ombre au texte et aux éléments.

La propriété **text-shadow** (resp. **box-shadow**) ajoute des ombres au texte (resp. à la boîte d'un élément). Elle accepte une liste d'ombres (séparées par des virgules) à appliquer au texte.

Chaque ombre est décrite par une certaine combinaison de **décalages X et Y** de l'élément (obligatoire), de **rayon de flou** et de **couleur**.

Note:

- La couleur de l'ombre. Elle peut être spécifiée avant ou après les valeurs de décalage.
- Si une **border-radius** est définie sur l'élément, l'ombre prendra les mêmes arrondis.

21. Les boutons en CSS

Les boutons peuvent être styliser à l'aide de plusieurs propriétés CSS.

Propriété	description	Exemple
background-color	modifier la couleur d'arrière-plan d'un bouton	{background-color: #008CBA;}
font-size	modifier la taille de la police d'un bouton	{font-size: 10px;}
Width	modifier la largeur d'un bouton	{width: 250px;}
Padding	modifier le remplissage d'un bouton	{padding: 10px 24px;}
border-radius	ajouter des coins arrondis à un bouton	{border-radius: 12px;}
Border	ajouter une bordure colorée à un bouton	{border: 2px solid #4CAF50;}
transition-duration Avec le selecteur :hover	pour déterminer la vitesse de l'effet "survol" (modifier le style d'un bouton lorsque vous déplacez la souris dessus.)	.button { transition-duration: 0.4s;} .button:hover { background-color: #4CAF50; color: white;}
box-shadow	pour ajouter des ombres à un bouton	{ box-shadow: 0 8px 16px 0 grey, 0 6px 20px 0 red;}
Opacity	pour ajouter de la transparence à un bouton (crée un aspect «désactivé»).	{ opacity: 0.6; cursor: not-allowed;}
Float	pour créer un groupe de boutons (côte à côte)	{ float: left;}
display:block	pour regrouper les boutons les uns sous les autres	{ display: block;}

22. Colonnes multiples CSS

CSS permet de définir facilement plusieurs colonnes de texte (comme dans les journaux).

Propriétés	Description	Exemple
column-count	spécifie le nombre de colonnes dans lesquelles un élément doit être divisé.	div { column-count: 3;}
column-gap	spécifie l'écart entre les colonnes.	div { column-gap: 40px;}
column-rule-style	spécifie le style de la règle entre les colonnes	div { column-rule-style: solid;}

column-rule-width	spécifie la largeur de la règle entre les colonnes	div { column-rule-width: 1px;}
column-rule-color	spécifie la couleur de la règle entre les colonnes	div { column-rule-color: lightblue;}
column-rule	propriété abrégée pour définir toutes les propriétés column-rule : la largeur, le style et la couleur de la règle entre les colonnes.	div { column-rule: 1px solid lightblue;}
column-span	spécifie le nombre de colonnes sur lesquelles un élément doit s'étendre.	h2 { column-span: all; } /*<h2> doit s'étendre sur toutes les colonnes*/

23. Responsive Web Design

Les pages Web peuvent être consultées à l'aide de nombreux appareils différents: ordinateurs de bureau, tablettes et téléphones. Votre page Web doit avoir une belle apparence et être facile à utiliser, quel que soit l'appareil.

La conception Web réactive est assurée en utilisant seulement du HTML et du CSS.

Types de médias CSS3 : all, print, screen, speech.

23.1. La fenêtre d'affichage

La fenêtre d'affichage varie en fonction de l'appareil et sera plus petite sur un téléphone mobile que sur un écran d'ordinateur. HTML5 a introduit une méthode permettant aux concepteurs Web de prendre le contrôle de la fenêtre, via la balise **<meta>**.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

23.2. La requête multimédia

Les requêtes multimédias sont une technique courante pour fournir une feuille de style personnalisée à différents appareils. (Introduite dans CSS3).

La règle **@media** est utilisé pour inclure un bloc de propriétés CSS uniquement si une certaine condition est vraie.

Elle peut être utilisées pour vérifier de nombreux éléments, tels que: **largeur et hauteur de la fenêtre, largeur et hauteur de l'appareil, l'orientation (la tablette / téléphone est-elle en mode paysage ou portrait?) et la résolution.**

Pour plus de détails consulter :

<https://developer.mozilla.org/fr/docs/Web/CSS>

<https://www.w3schools.com/css/default.asp>